

VLSI Group Project: Step-by-Step Guide to Complete All 10 Ideas

CSE 4327 | Built from your lab reports (Exp 1 to 7) and the Group Project handout

0. Read this first

- **What the handout requires:** one project per group, a report with Motivation, Methodology, Relevant window screenshots (each with a caption) and Results and conclusion, and one zip with all files (schematics, Verilog, test benches, scripts, reports). A changed title needs instructor approval. This guide covers all ten because you asked; each project can stand alone.
- **Status today:** #7 mostly done, #8 and #9 partly, #1 and #4 half, #2, #3, #5, #6 not started, #10 needs physical design (the handout says layout is excluded, so ask the instructor before starting it).
- **Every number in your reports must come from your own runs.** This guide gives methods and expected behaviour, not results. The only fixed result is the LFSR sequence in Project 6.
- **Suggested order (shared work first):** Virtuoso track 1 → 2 → 3 → 4 → 5 (each reuses cells from the one before). Verilog and synthesis track 7 → 8 → 6 → 9 → 10.
- **Screenshot habits:** crop out the VMware window frame, caption every figure in your lab-report style (Figure-1.1 and so on), save ADE states, and keep one project folder per idea.

1. Common toolbox

1.1 Virtuoso and ADE L (Projects 1 to 5)

1. Create a library attached to gpdk090 (same as Exp 1 to 3). Copy earlier cells with Library Manager (Edit → Copy) instead of redrawing them.
2. In ADE L: Setup → Model Libraries (gpdk090 Spectre models), choose the tran analysis (and dc where needed), and select outputs from the schematic.
3. Test bench standard: vdc 1.8 V on VDD, gnd on VSS, vpulse inputs 0 to 1.8 V with 50 to 100 ps rise and fall times, and a 10 fF load capacitor (analogLib cap) on every output so delays are comparable across circuits.

4. Propagation delay: measure at the 50 percent level, 0.9 V. t_{pHL} = time V_{out} falls through 0.9 V minus time V_{in} crosses 0.9 V; t_{pLH} is the rising equivalent. Report $t_p = (t_{pHL} + t_{pLH}) / 2$. Use the Calculator (Special Functions → delay) or two markers in Visualization XL. Rise and fall time are measured between 10 and 90 percent: 0.18 V to 1.62 V.
5. Parametric sweeps: Variables → Copy From Cellview, give each variable a value, then Tools → Parametric Analysis.
6. Save an ADE state (Session → Save State) for every test so any figure can be reproduced.

1.2 Verilog, NCLaunch, SimVision (Projects 6, 7, 9)

1. Same folders and flow as Exp 3 to 6: lab/simulation, cds.lib, NCLaunch: ncvlog → ncelab → ncsim.
2. Write self-checking test benches this time: keep an integer errors counter, compare outputs with expected values, print PASS or FAIL at the end, then \$finish. Keep \$monitor as well so the console log makes a good screenshot.
3. Screenshots per design: source, test bench, NCLaunch window with no errors, waveform with every port, console log.

1.3 RTL Compiler (Projects 7 to 10)

1. Folder layout from Exp 6: rtl, lib (slow.lib copied from the FoundationFlows GPDK045 timing folder), constraints, synthesis, simulation.
2. Use this script for every run. It adds the area and gates reports that your Exp 6 and Exp 7 runs did not have:

```

set_attr lib_search_path ../lib/
set_attr hdl_search_path ../rtl/
set_attr library slow.lib
read_hdl -v2001 TOP.v
elaborate TOP
read_sdc ../constraints/TOP.g
synthesize -to_mapped -effort medium
report timing > TOP_timing.rpt
report power > TOP_power.rpt
report area > TOP_area.rpt
report gates > TOP_gates.rpt
write_hdl > TOP_netlist.v
write_sdc > TOP.sdc

```

Replace TOP with your module name. If a redirected report fails on your version, run the report at the rc prompt and screenshot it instead.

3. Every synthesis table should contain: clock period, worst slack, cell count, area, leakage, dynamic and total power.
4. Run gui_show for the schematic screenshot.

Project 1: CMOS Standard Cell Library (INV, NAND, NOR, AND, OR, XOR)

Already done: INV, NAND, AND (Exp 1 to 3). **Missing:** NOR, OR, XOR, all truth-table checks in one place, and propagation delays.

1. Create library StdCells and copy your INV, NAND and AND cells into it.
2. Choose one sizing rule for all gates and state it in the report. Option A: keep your lab sizes (Wp 720 nm, Wn 240 nm, L 100 nm) everywhere, consistent with Exp 2 and 3. Option B: scale the stacks (NAND2 Wn 480 nm, NOR2 Wp 1440 nm), which gives fairer delays.
3. NOR2: two PMOS in series (VDD to output), two NMOS in parallel (output to VSS). Pins A, B, Y, VDD, VSS. Symbol: OR shape with an output bubble.
4. OR2: NOR2 symbol followed by your INV symbol (hierarchical, like AND = NAND + INV).
5. XOR2: four NAND2 symbols. With $n = \text{NAND}(A,B)$: $Y = \text{NAND}(\text{NAND}(A,n), \text{NAND}(B,n))$. That is 16 transistors.

6. For each gate: Check and Save, Create → Cellview → From Cellview, and replace the default shape with the standard symbol.
7. Truth-table test bench per gate: A = vpulse (period 20 ns, width 10 ns), B = vpulse (period 40 ns, width 20 ns), 10 fF load, tran 80 ns. Build the table: A, B, expected Y, simulated Y in volts, logic level.
8. Delay test per gate and per input arc: hold the other input at its non-controlling value (1 for NAND and AND, 0 for NOR and OR; for XOR run it twice, other input 0 and then 1). Pulse the tested input (period 10 ns, width 5 ns, 50 ps edges) and measure tpLH and tpHL as in Section 1.1.
9. Results table: gate, transistor count, tpLH, tpHL, tp, worst-case arc. Predict before measuring (INV fastest, AND and OR slower because they are two stages, XOR slowest with three NAND levels) and say whether your data agree.

Report: Motivation (these cells are what rc maps to in Exp 6 and 7), methodology, schematics and symbols, waveforms with truth tables, delay table, conclusion.

Project 2: Effect of PMOS/NMOS Sizing on Inverter Performance

Missing: everything beyond the single fixed 3:1 design.

1. Copy your NOT_gate inverter into a new library. Keep Wn = 240 nm and L = 100 nm and sweep Wp = 240, 360, 480, 600, 720, 960, 1200 nm (ratio 1 to 5).
2. Make Wp a variable: type wp in the PMOS Total Width field, define wp in ADE L (Copy From Cellview), and use Parametric Analysis. If the device does not accept a variable in that field (the finger-width callback can interfere), do one run per width by editing the PMOS and saving each result under its own name.
3. DC test: sweep Vin from 0 to 1.8 V. Measure VM where Vout = Vin (Calculator cross() on Vout minus Vin, or a marker at the intersection).
4. Transient test: Vin pulse 0 to 1.8 V, 100 ps edges, 10 fF load, period 10 ns. Measure rise time, fall time (10 to 90 percent), tpLH and tpHL.
5. Table: Wp/Wn, VM, tr, tf, tpLH, tpHL, tp, and relative area (Wp + Wn).
6. Plots: VTC family on one axis, VM vs ratio, tr and tf vs ratio, tp vs ratio.
7. Pick the best ratio and justify it with three criteria: VM closest to 0.9 V, tr close to tf, lowest tp. Mention the area cost. Report whichever ratio your data support, even if it is not 3:1.
8. Optional: sweep Wn at fixed Wp, or repeat at a second load (5 fF and 20 fF).

Report: theory (hole mobility, why symmetry matters), sweep method, plots, table, justification.

Project 3: Transistor-Level Full Adder and 4-bit Ripple-Carry Adder

Already have: Verilog adders (Exp 4) for expected values, and your NAND and INV cells.

Missing: all transistor-level work.

1. Build the 1-bit adder from your symbols: $p = A \text{ XOR } B$ (4 NAND), $\text{Sum} = p \text{ XOR } \text{Cin}$ (4 NAND), $\text{Cout} = \text{NAND}(\text{NAND}(A,B), \text{NAND}(\text{Cin},p))$. Total 11 NAND = 44 transistors. Pins A, B, Cin, Sum, Cout, VDD, VSS. Optional bonus: the 28-transistor mirror adder for a transistor-count comparison.
2. Create the symbol FA.
3. 1-bit test bench: Cin = vpulse (delay 10n, width 10n, period 20n), B = vpulse (delay 20n, width 20n, period 40n), A = vpulse (delay 40n, width 40n, period 80n), 10 fF on Sum and Cout, tran 80 ns. This counts abc = 000 to 111 in order. Expected Sum 0,1,1,0,1,0,0,1 and Cout 0,0,0,1,0,1,1,1 (matches your Exp 4 table).
4. 4-bit adder: new schematic with four FA symbols. Cout of stage i goes to Cin of stage i+1, and Cin of stage 0 is an input. Label A0 to A3, B0 to B3, S0 to S3.
5. Functional test with the six vectors from your Exp 4 4-bit test bench (2+3, 5+4+1, 7+1, 8+8, F+F+1, 0+0): drive each bit with a vpwl source, or set vdc values per run. Compare sums and carry with your Verilog waveforms.
6. Worst-case carry delay: A = 1111 (vdc 1.8 V on each bit), B = 0000, and Cin pulses 0 to 1.8 V, so the carry must travel through all four stages. Measure Cin to Cout of stage 3 and Cin to S3 (50 percent points). Also measure one stage alone and check that the 4-bit delay is roughly four stage delays.
7. Table: stage delay, 4-bit worst-case delay, transistor count.

Report: ripple-carry theory, schematic hierarchy screenshots, truth-table waveforms, worst-case delay waveform with markers, cross-check against Exp 4.

Project 4: Transistor-Level 2:1 and 4:1 Mux, Cross-Checked with Verilog

Already have: mux21 and mux41 in Verilog (Exp 3). **Missing:** the transistor-level designs and the comparison.

1. 2:1 mux: $Y = \text{NAND}(\text{NAND}(I_0, S'), \text{NAND}(I_1, S))$ with one INV for S' . 3 NAND + 1 INV = 14 transistors. Optional comparison: a transmission-gate version with 2 TG + 1 INV (6 transistors).
2. Create symbol MUX21 with pins I_0, I_1, S, Y, VDD, VSS .
3. 4:1 mux: three MUX21 symbols. Two first-level muxes on S_0 (I_0/I_1 and I_2/I_3), one second-level mux on S_1 .
4. Identical inputs for both simulations (binary-count pattern): input k ($k = 0$ for I_0 , then I_1 and so on, with the select lines after the data lines) toggles every 10×2^k ns. In Spectre use vpulse with v_1 0, v_2 1.8, delay = 10×2^k ns, width = 10×2^k ns, period = 20×2^k ns. The 2:1 mux has 3 inputs (run 80 ns); the 4:1 mux has 6 inputs (run 640 ns).
5. Matching Verilog test bench: all regs start at 0, one always block per input, for example always $\#(10 \times (2^{**k}))$ in_k = ~in_k; run the same time and \$monitor the output.
6. Compare: in Spectre read Vout 5 ns into every 10 ns slot (Calculator value(), or Results → Print → Transient Waveforms to export). Table columns: slot, inputs, Verilog out, Spectre Vout, logic level, match. Report the match rate and any glitch visible in the analog plot that zero-delay Verilog cannot show.
7. Add delays: S to Y and I to Y for the 2:1 mux.

Report: both waveform sets side by side, comparison table, glitch discussion (zero-delay RTL vs real gate delay).

Project 5: Transistor-Level D Latch and D Flip-Flop (clock-to-Q and setup time)

Already have: Verilog d_ff (Exp 5) for the expected behaviour. **Missing:** all transistor-level work.

1. Transmission gate (TG) cell: NMOS and PMOS in parallel, pins IN, OUT, C (NMOS gate), CB (PMOS gate).
2. D latch (transparent when $\text{CLK} = 1$): TG_in ($C = \text{CLK}$, $\text{CB} = \text{CLKb}$) from D to node X; $X \rightarrow \text{INV1} \rightarrow \text{QB} \rightarrow \text{INV2} \rightarrow \text{Q}$; feedback TG_fb from Q back to X with $C = \text{CLKb}$ and $\text{CB} = \text{CLK}$. One INV makes CLKb. 10 transistors.
3. Master-slave D flip-flop: master latch transparent when $\text{CLK} = 0$ (swap C and CB on its TGs), slave latch transparent when $\text{CLK} = 1$, master Q feeds slave D. Rising-edge triggered, about 18 transistors. Make symbols DLATCH and DFF.

4. Functional test: CLK vpulse (period 10 ns, width 5 ns), D toggling away from the edges. Show that the latch follows D while CLK = 1 but the flip-flop changes only at rising edges.
5. Clock-to-Q: D stable 2 ns before the edge, 10 fF on Q. tcq = Q 50 percent time minus CLK 50 percent time, for Q rising (D = 1) and Q falling (D = 0).
6. Setup time: fix the CLK rising edge at tclk (for example 5 ns). Drive D with one rising step whose delay is the variable td_d. Parametric sweep td_d from tclk minus 1 ns to tclk plus 0.2 ns in 20 ps steps. For each run note whether Q captured 1 at tclk plus 2 ns. Setup time = tclk minus the latest td_d that is still captured (refine the sweep to 10 ps near the boundary). Repeat for a falling D. Optional: define setup as the point where tcq grows by 10 percent, and plot tcq against setup margin.
7. Optional: hold time, by sweeping D changes after the edge.
8. Table: tcq (LH), tcq (HL), tsetup (rise), tsetup (fall), transistor count.

Report: circuit explanation, schematics, functional waveforms, sweep family plot, tables, comparison with the ideal zero-delay Verilog d_ff.

Project 6: Shift Register and LFSR Pseudo-Random Generator (Verilog)

Already have: d_ff (Exp 5). **Missing:** both designs.

1. Shift register, structural: four d_ff instances, q0 takes din, each next d is the previous q, common clk and rst. Also write the behavioural version, $q \leq \{q[2:0], \text{din}\}$, and check that both give identical waveforms.
2. Test bench: reset, then shift in 1,0,1,1 (one bit per clock) and show the q bits after each edge; run 4 more clocks to flush. Show the SIPO view ($q[3:0]$) and the SISO view (q3).
3. LFSR: 4-bit Fibonacci, shift left, feedback = $q[3] \wedge q[2]$ (polynomial $x^4 + x^3 + 1$), reset to seed 0001:

```
module lfsr4 (input clk, input rst, output reg [3:0] q);
  always @(posedge clk or negedge rst) begin
    if (!rst) q <= 4'b0001;
    else q <= {q[2:0], q[3] ^ q[2]};
  end
endmodule
```

4. Expected sequence (derive it by hand in the report first, then confirm in simulation): 0001, 0010, 0100, 1001, 0011, 0110, 1101, 1010, 0101, 1011, 0111, 1111, 1110, 1100, 1000, then back to 0001. Length $15 = 2^4 - 1$ (maximal).
5. Self-checking test bench: save the seed, clock 15 times, check q returns to the seed and that no state repeated earlier (track visited states in a 16-bit mask). Run 40 clocks to show it repeats every 15.
6. Show the lock-up state: force q to 0000 and show it stays at 0000. Explain why an XOR-feedback LFSR needs a non-zero seed.
7. Optional: 8-bit LFSR with taps at bits 8, 6, 5, 4 (feedback $q[7] \oplus q[5] \oplus q[4] \oplus q[3]$). The period should be 255, so count it in the test bench. Mention applications (test pattern generation, BIST, scrambling, pseudo-random numbers).

Report: theory, state-transition table, waveforms (mark where the sequence repeats), period measurement.

Project 7: Programmable 4-bit Up/Down Counter (simulation and synthesis)

Already have: up-counter simulated (Exp 5) and synthesized (Exp 6). **Missing:** the up/down control, a load input if you want it programmable, area reports, and a comparison with the plain up-counter.

1. RTL, saved as updown_counter.v (tie load to 0 if your instructor only wants up/down plus reset):

```
`timescale 1ns/1ps
module updown_counter (
    input clk, input rst, input up_down, input load,
    input [3:0] din, output reg [3:0] count
);
    always @(posedge clk or negedge rst) begin
        if (!rst) count <= 4'b0000;
        else if (load) count <= din;
        else if (up_down) count <= count + 1'b1;
        else count <= count - 1'b1;
    end
endmodule
```

2. Self-checking test bench. The reference model is written differently from the RTL (modulo arithmetic), checked half a cycle after each rising edge:

```

`timescale 1ns/1ps
module updown_counter_test;
    reg clk = 0, rst = 0, up_down = 1, load = 0;
    reg [3:0] din = 4'b0000;
    wire [3:0] count;
    integer expected = 0, errors = 0;
    updown_counter uut (.clk(clk), .rst(rst), .up_down(up_down),
                       .load(load), .din(din), .count(count));
    always #5 clk = ~clk;
    always @(posedge clk or negedge rst) begin
        if (!rst) expected = 0;
        else if (load) expected = din;
        else expected = (expected + (up_down ? 1 : 15)) % 16;
    end
    always @(negedge clk) if (rst && count !== expected[3:0]) begin
        errors = errors + 1;
        $display("MISMATCH t=%0t count=%0d expected=%0d", $time, count,
expected);
    end
    initial begin
        $monitor("t=%0t rst=%b ud=%b ld=%b din=%0d count=%0d",
                $time, rst, up_down, load, din, count);
        #12 rst = 1;
        repeat (20) @(negedge clk);           // up, through the 15 to 0 wrap
        up_down = 0;
        repeat (20) @(negedge clk);           // down, through the 0 to 15 wrap
        din = 4'd10; load = 1; @(negedge clk); load = 0; // load 10
        repeat (6) @(negedge clk);
        repeat (8) begin up_down = ~up_down; @(negedge clk); end // direction
toggling
        #3 rst = 0; #1 if (count !== 0) errors = errors + 1; // asynchronous
reset
        #5 rst = 1;
        repeat (4) @(negedge clk);
        if (errors == 0) $display("PASS"); else $display("FAIL: %0d errors",
errors);
        $finish;
    end
endmodule

```

3. Run the NCLaunch flow. Screenshots: NCLaunch with no errors, console ending in PASS, and a waveform with clk, rst, up_down, load, din and count annotated by phase (reset, up with wrap, down with wrap, load, direction toggling, asynchronous reset).
4. Constraints (updown_counter.g). Your Exp 6 file constrained only rst; every new input needs an input delay or its paths stay unconstrained:

```
create_clock -name clk -period 10 -waveform {0 5} [get_ports "clk"]
set_clock_transition -rise 0.1 [get_clocks "clk"]
set_clock_transition -fall 0.1 [get_clocks "clk"]
set_clock_uncertainty 0.1 [get_clocks "clk"]
set_input_delay -max 1.0 [get_ports {rst up_down load din}] -clock
[get_clocks "clk"]
set_output_delay -max 1.0 [get_ports "count"] -clock [get_clocks "clk"]
```

5. Synthesize with the Section 1.3 script. Collect timing, power, area and gates reports, and the gui_show schematic. In the schematic point out the add/subtract logic and the load multiplexing in front of the flip-flops.
6. Comparison table against your Exp 6 up-counter: design, cell count, area, worst slack, leakage, dynamic, total power. State what the up/down and load features cost.
7. Optional: gate-level simulation of the netlist with the same test bench. It needs the Verilog models of the gsclib045 cells; ask the instructor where they are.

Report: motivation (programmable counters in timers and address generators), RTL and test bench listings, simulation evidence, SDC and script, synthesis table, schematic, conclusion.

Project 8: Impact of Clock Period Constraints on Synthesis Results

Already have: a working synthesis flow (Exp 6, 7). **Missing:** the sweep and the analysis.

1. Design under test: the Project 7 counter, plus (recommended) the registered 32-bit adder from Project 9. The counter has a very short path, so the adder shows the trade-off far more clearly.
2. Periods in ns: 10, 5, 3, 2, 1.5, 1.2, 1.0, 0.8, 0.6, 0.5. Keep going down until slack goes negative.
3. Create a constraints template TOP_template.g. Scale the input and output delays with the period (10 percent). A fixed 1.0 ns delay makes the tight periods impossible and would hide the real trade-off. Edit the port names for your design:

```

create_clock -name clk -period @P@ -waveform {0 @H@} [get_ports "clk"]
set_clock_transition -rise 0.1 [get_clocks "clk"]
set_clock_transition -fall 0.1 [get_clocks "clk"]
set_clock_uncertainty 0.1 [get_clocks "clk"]
set_input_delay -max @D@ [get_ports {rst up_down load din}] -clock
[get_clocks "clk"]
set_output_delay -max @D@ [get_ports "count"] -clock [get_clocks "clk"]

```

4. Copy the Section 1.3 script to rc_template.tcl and put @P@ in every output file name, for example read_sdc ../constraints/TOP@P@.g and report timing > TOP_timing_@P@.rpt. Then run all periods with one csh loop from the synthesis folder:

```

foreach P (10 5 3 2 1.5 1.2 1.0 0.8 0.6 0.5)
  set H = `echo $P | awk '{print $1/2}'`
  set D = `echo $P | awk '{print $1*0.1}'`
  sed -e "s/@P@/$P/g" -e "s/@H@/$H/g" -e "s/@D@/$D/g"
  ../constraints/TOP_template.g > ../constraints/TOP_$P.g
  sed -e "s/@P@/$P/g" rc_template.tcl > rc_$P.tcl
  echo exit >> rc_$P.tcl
  rc -f rc_$P.tcl
end

```

5. Fill one table row per period: period, frequency, worst slack, timing met (slack of 0 or more), cell count, area, leakage, dynamic and total power. Pull values from the report files (grep for slack and read the area and power lines).
6. Minimum period: Tmin is roughly period minus slack at the tightest period that still meets timing. Confirm with finer steps around it.
7. Plots: slack vs period, area vs period, power vs frequency, and optionally area x delay.
8. Explain the trade-offs from your data: at relaxed clocks the tool keeps minimum-size cells and the slack is wasted; near the limit it upsizes cells, adds buffers and restructures logic, so area and power rise; dynamic power also scales with frequency ($f \times C \times V^2$). Mention that rc estimates power with default switching activity, so use the numbers for comparison, not as silicon predictions.
9. Optional: repeat one period at effort low, medium and high.

Report: motivation (timing closure vs area and power), the script and loop, the table, the four plots, trade-off discussion.

Project 9: Adder Architecture Comparison (ripple-carry vs carry-lookahead vs pipelined, 32-bit)

Already have: the pipelined 32-bit adder (Exp 7) and behavioural adders (Exp 4, which let the simulator and synthesizer choose the structure). **Missing:** structural ripple-carry, carry-lookahead, and the same-constraint comparison.

1. Ripple-carry, structural (rca32.v). Do not use the plus operator here; the point is an explicit carry chain:

```
module fa (input a, input b, input cin, output sum, output cout);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule

module rca32 (input [31:0] a, input [31:0] b, input cin,
              output [31:0] sum, output cout);
    wire [32:0] c;
    assign c[0] = cin;
    genvar i;
    generate
        for (i = 0; i < 32; i = i + 1) begin : stage
            fa u (.a(a[i]), .b(b[i]), .cin(c[i]), .sum(sum[i]), .cout(c[i+1]));
        end
    endgenerate
    assign cout = c[32];
endmodule
```

2. Carry-lookahead (cla32.v): eight 4-bit lookahead blocks, with the block carry rippling between blocks. Generate $g = a \& b$ and propagate $p = a \wedge b$, and $c[i+1] = g[i] \mid (p[i] \& c[i])$, expanded inside each block:

```
module cla4 (input [3:0] a, input [3:0] b, input cin,
             output [3:0] sum, output cout);
    wire [3:0] g = a & b;
    wire [3:0] p = a ^ b;
    wire [4:0] c;
    assign c[0] = cin;
    assign c[1] = g[0] | (p[0] & c[0]);
    assign c[2] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & c[0]);
    assign c[3] = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0])
                | (p[2] & p[1] & p[0] & c[0]);
endmodule
```

```

    assign c[4] = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1])
                | (p[3] & p[2] & p[1] & g[0]) | (p[3] & p[2] & p[1] & p[0] &
c[0]);
    assign sum  = p ^ c[3:0];
    assign cout = c[4];
endmodule

module cla32 (input [31:0] a, input [31:0] b, input cin,
              output [31:0] sum, output cout);
    wire [8:0] c;
    assign c[0] = cin;
    genvar i;
    generate
        for (i = 0; i < 8; i = i + 1) begin : blk
            cla4 u (.a(a[4*i+3:4*i]), .b(b[4*i+3:4*i]), .cin(c[i]),
                .sum(sum[4*i+3:4*i]), .cout(c[i+1]));
        end
    endgenerate
    assign cout = c[8];
endmodule

```

Optional second level: compute block group generate and propagate and look ahead across blocks for a full two-level CLA.

3. Self-checking test bench for rca32 and cla32 side by side: corner cases (0+0, FFFFFFFF+1, 7FFFFFFF+1, 80000000+80000000, FFFFFFFF+FFFFFFF+1) plus 1000 random vectors from \$random. Expected value: $\text{exp} = \{1'b0,a\} + \{1'b0,b\} + \text{cin}$ as a 33-bit reg, compared with {cout,sum}. Print PASS or FAIL.
4. Pipelined adder: rerun your Exp 7 test bench for longer than 90 ns so the 9-deep reference chain fills, and show errors = 0 with a PASS. Your 80 ns run ended with errors = 7 and no PASS.
5. For a fair timing comparison, wrap the combinational adders in input and output registers (latency 2 clocks). The pipelined adder already has registers (latency 9 clocks):

```

module rca32_reg (input clk, input rst, input [31:0] a, input [31:0] b,
                  input cin, output reg [31:0] sum, output reg cout);
    reg [31:0] a_r, b_r; reg cin_r;
    wire [31:0] s; wire co;
    rca32 core (.a(a_r), .b(b_r), .cin(cin_r), .sum(s), .cout(co));
    always @(posedge clk or negedge rst) begin
        if (!rst) begin a_r <= 0; b_r <= 0; cin_r <= 0; sum <= 0; cout <= 0; end
        else begin a_r <= a; b_r <= b; cin_r <= cin; sum <= s; cout <= co; end
    end
endmodule

```

Make cla32_reg the same way. Verify each wrapper with the self-checking bench (compare after 2 clocks).

6. Synthesize all three with identical settings: same slow.lib, same SDC (use the Project 8 template and loop), same script. At a relaxed clock (10 ns) all three meet timing and differences are small, so also sweep tight periods until each one fails.
7. Table: architecture, latency in clocks, minimum period, maximum frequency, results per second (one result per clock for all three), time to first result (latency x period), area, cell count, flip-flop count, power at the same clock.
8. Discuss: ripple-carry has the smallest area and the longest path; carry-lookahead shortens the path at the cost of extra gates; the pipelined adder reaches the highest frequency at the cost of many flip-flops, more power, and 9 clocks of latency. Note that rc may restructure arithmetic, so architecture differences can shrink at relaxed clocks; say so if your data show it. Optional fourth data point: the behavioural $a + b + \text{cin}$ adder, showing what the tool builds by itself.

Report: motivation (speed vs area vs power in datapath design), the three RTL listings, verification evidence, synthesis table, critical-path screenshots, comparison charts, conclusion.

Project 10: Complete RTL-to-GDSII Flow of an 8-bit Counter

Already have: RTL, simulation, SDC, rc synthesis, netlist (Exp 5, 6). **Missing:** the whole physical design.

Before starting: this needs Cadence Encounter Digital Implementation (EDI) and the GPDK045 physical libraries, and the handout says layout design is excluded from the course. Get instructor approval, and ask for the GPDK045 EDI example scripts. They are normally in the same FoundationFlows folder you copied slow.lib from (look under

EXAMPLES/EDI/DESIGN/GPDK). Command and menu names below follow common EDI usage but differ between versions, so use the GUI menu equivalents and keep the encounter command and log files as evidence.

1. RTL: counter_8bit.v with the same style as Exp 6, plus a self-checking test bench that runs past the 255 to 0 wrap and prints PASS.
2. Synthesis with rc (Section 1.3 script and a 10 ns SDC). Outputs: counter_8bit_netlist.v and counter_8bit.sdc.
3. Collect from the GPDK045 package: technology LEF, standard-cell (macro) LEF, timing libraries (slow, and fast if provided), capacitance table or QRC files if provided, standard-cell GDS and a stream map file. Note the core site name in the LEF.
4. Start EDI in csh (command encounter) and import the design (File → Import Design): netlist, top cell, LEF files, power net VDD, ground net VSS, timing setup using the library views and the synthesis SDC.
5. Floorplan (Floorplan → Specify Floorplan): aspect ratio 1, core utilization about 0.6 to 0.7, core-to-boundary spacing about 5 to 10 um. Place or auto-place the pins.
6. Power plan: Add Rings (VDD and VSS on the top two metal layers your LEF allows), Add Stripes, then Route → Special Route to connect the standard-cell power rails.
7. Placement (Place → Standard Cells). Save the design, then run timing (timeDesign pre-CTS) and an optimisation pass (optDesign pre-CTS). Record area and utilization.
8. Clock tree: create the clock tree spec and synthesize the clock tree (clockDesign). Record skew and insertion delay, then timeDesign post-CTS for setup and hold, and optDesign post-CTS.
9. Routing: Route → NanoRoute (routeDesign), then optDesign post-route and timeDesign post-route (setup and hold).
10. Fillers: Place → Physical Cell → Add Filler, using the filler cells from the LEF, largest first.
11. Verification: Verify Geometry (DRC), Verify Connectivity, Verify Process Antenna. Target zero violations; if any appear, fix and rerun until clean.
12. Export: Stream Out to counter_8bit.gds with the stream map and the standard-cell GDS merged in (matching units). Also save the final netlist, DEF and SDF, and save the design database.
13. Screenshots per stage: floorplan, power grid, placement, clock tree, final routed layout, and the verification reports.
14. Stage-metrics table: stage, setup WNS, hold WNS, area, utilization, cell count, DRC violations. Add clock skew and the size of the GDS file.

Report: motivation (the complete digital flow), methodology (the 14 steps with the exact settings you used), screenshots per stage, metrics table, conclusion with the problems you hit and how you fixed them. Put the log and command files in the zip.

2. Report template (use for every project)

1. **Cover page:** the same UIU lab-report cover as your earlier reports, with the project title and your team details.
2. **Motivation:** one paragraph on why the project was chosen and which concept it addresses.
3. **Methodology:** tools and versions, library (gpdk090 or gpdk045), every design parameter (W/L, load, clock period, constraints), and the numbered flow you followed.
4. **Relevant window screenshots:** schematics, symbols, Verilog, test benches, NCLaunch and SimVision windows, ADE and Visualization windows, rc output, tool reports. Every figure gets a caption in your lab style (Figure-1.1 and so on). Crop the VMware frame.
5. **Results and conclusion:** results tables and plots, comparison with what theory predicts, limitations, and what you learned.

Zip layout (one zip, submitted through the form link in the handout): report, schematics (library export or screenshots plus netlists), verilog (rtl and test benches), constraints, scripts, reports (rc timing, power, area, gates), waveforms, screenshots.

3. Pitfalls seen in your earlier runs

- **File and module names:** keep them identical across simulation and synthesis (your Exp 6 used 4-bit_counter.v and counter_4bit, then cntr4bit for synthesis). Pick one name per design.
- **Run long enough:** simulate past the pipeline latency or the wrap point before taking the waveform screenshot, and show PASS in the console.
- **Constrain every port:** a new input without set_input_delay is silently unconstrained.
- **Fair comparisons:** same load capacitor for every gate, same SDC and script for every architecture, same stimulus for Verilog and Spectre.
- **Units:** SimVision times are in ps, Spectre plots in ns; label axes in screenshots and tables.
- **Do not copy results:** use your own run numbers in every table, and record which run produced them.